

Tema 3

Plotly y Bokeh en la práctica

Jorge Dueñas Lerin

jorge.duenas.lerin@upm.es

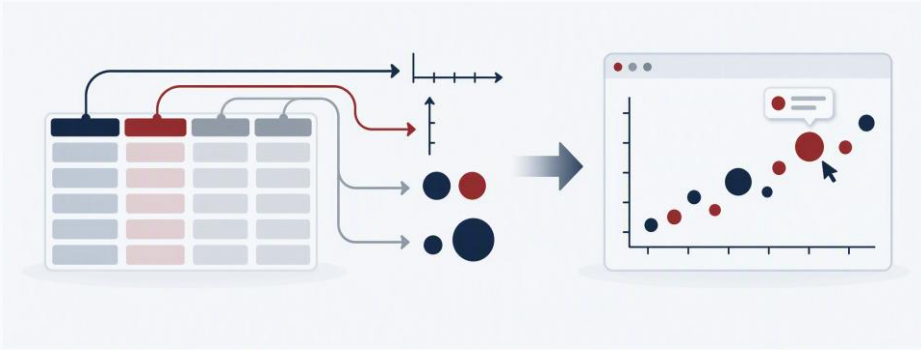
El hilo de hoy: un mismo gráfico, dos formas de construirlo

Plotly: describir el gráfico

Cómo: dices qué columna va a cada canal (x, y, color, size) y la librería resuelve el resto

Resultado: una figura (trazas + layout) que se ajusta con **update_traces** y **update_layout**

Interactividad: viene de serie: hover, zoom, leyenda y animación

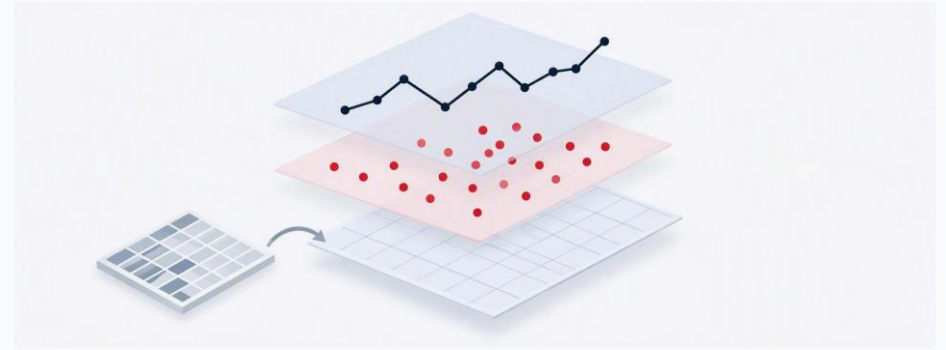


Bokeh: construir el gráfico

Cómo: creas un lienzo con `figure()` y añades capas (glyphs) una a una

Datos: **ColumnDataSource** conecta Python con el navegador y **comparte** selecciones

Interactividad: se declara en tools, HoverTool, Slider y CustomJS



Índice

1. Plotly Express: describir el gráfico
2. La figura Plotly por dentro
3. Bokeh: construir capa a capa
4. Elegir, practicar y seguir explorando

Índice

1. Plotly Express: describir el gráfico
2. La figura Plotly por dentro
3. Bokeh: construir capa a capa
4. Elegir, practicar y seguir explorando

1. Plotly Express: describir el gráfico

Plotly Express: describir el gráfico con una sola llamada

Cómo piensa Plotly Express

Columnas → **canales**: x, y, color, size y hover_name reciben nombres de columna

Una función por gráfico: px.scatter, px.line, px.bar, px.histogram... con los mismos argumentos

Interactividad de serie: zoom, pan, leyenda que aísla series y descarga en PNG

Tiempo: animation_frame convierte una columna en slider y botón Play



Qué obtienes

Una figura: px devuelve una go.Figure que puedes seguir ajustando

Un HTML autónomo: fig.write_html() se abre en cualquier navegador

Vista inmediata: fig.show() desde un script o un cuaderno Jupyter

Base para apps: la misma figura se reutiliza en Dash



1. Plotly Express: describir el gráfico

Del gráfico estático al dinámico: filtrar y explorar

```
# 01_01_02_dinamico_2007.py
import plotly.express as px
df = px.data.gapminder().query("year == 2007") # Filtramos
# Exploración interactiva
fig = px.scatter(
    df, .
    x="gdpPercap",
    y="lifeExp",
    color="continent",
    color_discrete_map=colores,
    log_x=True, # Escala log.
    hover_name="country",
    hover_data={
        "gdpPercap": ":", ".0f", "lifeExp": ":", ".1f", "pop": ":", ".0f"
    }, labels={
        "gdpPercap": "PIB per cápita (USD, escala logarítmica)",
        "lifeExp": "Esperanza de vida (años)",
        "continent": "Continente",
    },
)
fig.write_html(salidas / "01_01_02_dinamico_2007.html",
include_plotlyjs=True)
fig.show()
```

Qué aporta la interactividad

Overplotting: 1.704 puntos en un lienzo impiden seguir un país

Filtrar primero: query("year == 2007") reduce el ruido antes de dibujar

Detalle bajo demanda: hover_name y hover_data sustituyen las etiquetas

Escala log: log_x=True separa los países de PIB bajo

Colores fijos: color_discrete_map repite la paleta en todos los ejemplos

1. Plotly Express: describir el gráfico

Tooltips por defecto frente a tooltips útiles

```
# 01_06_01_hover.py - tooltip por defecto vs. tooltip util
df = px.data.gapminder().query("year == 2007")
# --- 2. Tooltip por defecto ---
fig_defecto = px.scatter(
    df, x="gdpPercap", y="lifeExp",
    color="continent", log_x=True,
    labels={"gdpPercap": "PIB per capita (USD, log)",
            "lifeExp": "Esperanza de vida (anos)"},
)
# --- 3. Tooltip util con tres datos interpretables ---
fig_util = px.scatter(
    df, x="gdpPercap", y="lifeExp",
    color="continent", log_x=True,
    hover_name="country",          # cabecera del tooltip
    hover_data={"pop": ":", ".0f", "gdpPercap": ":", ".0f",
                "lifeExp": ":", ".1f", "iso_alpha": False},
    labels={...},
)
fig_util.write_html(salidas / "01_06_01_hover_util.html")
fig_util.show()
# Error tipico: un tooltip con demasiados campos es una tabla.
```

Claves del tooltip útil

hover_name: el país como cabecera en negrita

hover_data como dict: formato por columna (':', '.0f', ':", '.1f')

False: oculta columnas técnicas como iso_alpha

labels=: nombres legibles en ejes, leyenda y tooltip

Regla: tres o cuatro campos que respondan a la pregunta

1. Plotly Express: describir el gráfico

La misma gramática para cualquier tipo de gráfico

```
# 02_01_misma_interfaz.py
# Misma gramática: cambia la función, no la forma de pensar
import plotly.express as px

df = px.data.gapminder()
europa = df.query("continent == 'Europe'")
d2007 = df.query("year == 2007")

# Línea: evolución temporal de cada país
fig1 = px.line(europa, x="year", y="lifeExp", color="country")

# Barras: los 10 países más poblados en 2007
fig2 = px.bar(d2007.nlargest(10, "pop"), x="country", y="pop")

# Distribución: histograma y caja por continente
fig3 = px.histogram(d2007, x="lifeExp", color="continent")
fig4 = px.box(d2007, x="continent", y="lifeExp", points="all")

for fig in (fig1, fig2, fig3, fig4):
    fig.update_layout(template="simple_white")
    fig.show()
```

La gramática de px

Mismos argumentos: x, y, color, hover_name y labels valen en todas las funciones

px.line: evolución temporal; combina bien con hovermode='x unified'

px.bar: comparar magnitudes entre categorías

px.histogram / px.box: distribuciones; points='all' muestra cada país

Y más: mapas y 3D siguen la misma lógica (galerías al final)

1. Plotly Express: describir el gráfico

Distribuciones marginales y small multiples

```
# 01_01_05_dinamico_margen.py
fig = px.scatter(
    ...
    #marginal_x="xxxxx", # "box", "violin" o "histogram"
    marginal_y="rug", # Añade el diagrama en el eje Y derecho
)

# 01_01_06_dinamico_facets.py
fig = px.scatter(
    ...
    color="continent",
    facet_col="continent", # Sub gráficos
    ...
)

fig.update_layout(
    legend=dict(itemclick=False, itemdoubleclick=False)
)
fig.for_each_annotation(
    lambda a: a.update(text=a.text.split("=")[-1])
)
```

Más contexto en una vista

marginal_x / marginal_y: distribución de cada eje junto al gráfico

facet_col: small multiples con ejes compartidos

Oclusión: separar continentes evita burbujas superpuestas

Leyenda fija: itemclick=False, cada panel ya es una categoría

for_each_annotation: limpia los títulos de los paneles

1. Plotly Express: describir el gráfico

Animaciones temporales en Plotly Express

01_01_04_dinamico_slider.py

```
fig = px.scatter(
    df,
    x="gdpPercap",
    y="lifeExp",
    animation_frame="year", # Genera el slider y la animación
    animation_group="country", # punto a conectar entre frames
    size="pop",
    ...
    # Fijar ejes. No es histograma, no es necesario comenzar en 0
    # Valores posibles. Se utiliza el espacio de forma eficiente
    # sin generar distorsión visual
    range_x=[100, 100000],
    range_y=[25, 90],
)
fig.write_html(
    salidas / "01_01_04_dinamico_slider.html",
    include_plotlyjs=True,
    auto_play=False
)
```

Claves de la animación

animation_frame: un fotograma por año, con slider y Play

animation_group: mantiene la identidad de cada país

Ejes fijos: range_x y range_y evitan saltos de escala

auto_play=False: el usuario decide cuándo reproducir

Uso responsable: animar solo si el cambio temporal es el mensaje

1. Plotly Express: describir el gráfico

Índice

1. Plotly Express: describir el gráfico
2. La figura Plotly por dentro
3. Bokeh: construir capa a capa
4. Elegir, practicar y seguir explorando

2. La figura Plotly por dentro

Anatomía de una figura Plotly: trazas + layout

```
# 02_02_anatomia.py anatomía de figura: data (trazas) + layout
df = px.data.gapminder().query("year == 2007")
fig = px.scatter(df, x="gdpPercap", y="lifeExp",
                 color="continent", log_x=True,
                 hover_name="country")

print(type(fig))          # plotly.graph_objs._figure.Figure
print(len(fig.data))      # 5 trazas: una por continente

# 1) Ajustar las trazas (fig.data)
fig.update_traces(
    marker=dict(size=9, opacity=0.8),
    hovertemplate="<b>{%{hovertext}</b><br>"
                  "PIB: {%x:,.0f} USD<br>"
                  "Vida: {%y:.1f} años<extra></extra>")

# 2) Ajustar el aspecto (fig.layout)
fig.update_layout(template="simple_white",
                  title="Gapminder 2007", hovermode="closest")
fig.update_xaxes(title="PIB per cápita (USD, log)")
fig.show()
```

Dos partes, tres métodos

fig.data: las trazas (puntos, líneas, barras); px crea una por color

fig.layout: ejes, título, leyenda, plantilla y modo de hover

update_traces: cambia todas las trazas a la vez: marcadores o hovertemplate

update_layout / update_xaxes: ajustan el aspecto sin rehacer el gráfico

hovertemplate: {%x:,.0f}, y <extra></extra> dan control total del tooltip

[Más sobre hover](#)

2. La figura Plotly por dentro

Bajar a graph_objects: menús de selección

```
# 01_01_03_dinamico_menu.py
import plotly.express as px
import plotly.graph_objects as go
fig = go.Figure()
# Añadimos una traza por cada año, solo visible primera (1952)
for anio in anios:
    df_año = df[df["year"] == anio]
    fig.add_trace(
        go.Scatter(
            x=df_año["gdpPercap"], y=df_año["lifeExp"],
            mode="markers", text=df_año["country"], # hover
            visible=(anio == anios[0]), # Solo True en el 1º
        )
    )
# Construimos los botones para el menú
botones = []
for i, anio in enumerate(anios):
    botones.append(dict(..., method="update", ...))
fig.update_layout(
    updatemenus=[..., buttons=botones],
    xaxis_type="log",
    xaxis_range=[2.0, 5.5],
)
```

Cómo funciona el menú

Figura = trazas + layout: go.Figure da control total

Una traza por año: solo una es visible en cada momento

updatemenus: botones que alternan la lista de visibilidad

method='update': cambia trazas y título a la vez

Ejes fijos en log: xaxis_range

2. La figura Plotly por dentro

Zoom temporal con rangelslider y rangeselector

```
# 02_03_zoom_slider.py
import pandas as pd
import plotly.express as px

# --- 1. Serie temporal de un pais ---
df = px.data.gapminder().query("country == 'Spain'").copy()
df["fecha"] = pd.to_datetime(df["year"].astype(str), format="%Y")

# --- 2. Controles de navegacion en el eje temporal ---
fig = px.line(
    df,
    x="fecha",
    y="lifeExp",
    markers=True,
    labels={"fecha": "Año", "lifeExp": "Esperanza de vida
(anos)"},
)
fig.update_xaxes(
    rangelslider_visible=True, # Mostrar deslizador
    rangeselector={ ... },
)
fig.show()
```

Navegar sin filtrar

Eje de fechas: `pd.to_datetime` convierte el año en fecha

rangelslider: desplaza el periodo sin perder el contexto

rangeselector: atajos a 15 años, 30 años o todo

Zoom nativo: arrastrar para ampliar, doble clic para volver

Nota: más útil si hay más detalle. Ejemplo tickers económicos

2. La figura Plotly por dentro

Índice

1. Plotly Express: describir el gráfico
2. La figura Plotly por dentro
3. Bokeh: construir capa a capa
4. Elegir, practicar y seguir explorando

3. Bokeh: construir capa a capa

Bokeh: las piezas de un gráfico

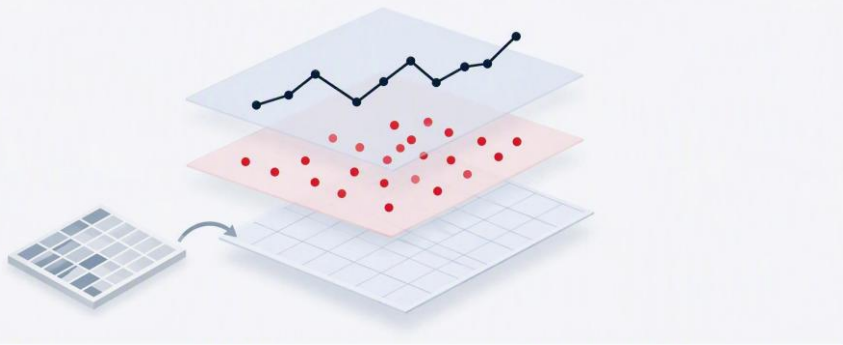
Cómo se construye

figure(): lienzo, con ejes, tamaño y herramientas (tools=)

Glyphs: capas como p.scatter, p.line, p.vbar, p.varea, etc

ColumnDataSource: los datos que viajan al navegador y comparten las vistas.

Interacción y salida: HoverTool y widgets; show() o save() generan el HTML.



¿Cuándo elegir Bokeh frente a Plotly?

Lógica propia sin servidor: CustomJS conecta widgets y gráficos en un HTML autónomo.

Vistas vinculadas: un ColumnDataSource compartido sincroniza la selección entre gráficos.

Construir por capas: glyphs, bandas y tendencias se superponen paso a paso.

Tooltips con lógica: CustomJSHover aplica reglas condicionales en la etiqueta.



3. Bokeh: construir capa a capa

ColumnDataSource y HoverTool en Bokeh

```
# 01_04_01_bokeh_hover.py
from bokeh.models import ColumnDataSource, HoverTool
from bokeh.plotting import figure, output_file, save, show
df = px.data.gapminder().query("year == 2007").copy()
df["color"] = df["continent"].map(COLORES)
df["tamano"] = (df["pop"] / df["pop"].max()) * AJUSTE + MIN

source = ColumnDataSource(df)
p = figure(width=850, height=500, x_axis_type="log",
          tools="pan,wheel_zoom,box_zoom,reset,save",
        )
p.scatter(
    x="gdpPercap", y="lifeExp", source=source,
    size="tamano", color="color",
)

# 3. Tooltip personalizado
p.add_tools(HoverTool(tooltips=[
    ("País", "@country"),
    ("PIB per cápita", "@gdpPercap{0,0} USD"),
    ("Esperanza de vida", "@lifeExp{0.0} años"),
]))
```

Puntos clave de Bokeh

Lienzo y capas: el gráfico es un lienzo `figure()` al que se añaden capas (glyphs).

ColumnDataSource: tabla de columnas con nombre que viaja al navegador. Los glyphs y tooltips leen de ella

tools: qué herramientas muestra la barra.

@col{formato} unidades

{0,0} miles

{0.0} un decimal

3. Bokeh: construir capa a capa

Capas y leyenda interactiva en Bokeh

02_04_bokeh_capas capas y leyenda

```
import plotly.express as px
from bokeh.palettes import Category10
from bokeh.plotting import figure, show

df = px.data.gapminder()
países = ["Spain", "China", "Brazil", "Nigeria"]

p = figure(width=850, height=450, x_axis_label="Año",
           y_axis_label="Esperanza de vida (años)")

for país, color in zip(países, Category10[4]):
    d = df[df["country"] == país]
    p.line(d["year"], d["lifeExp"], line_width=3,
           color=color, legend_label=país)      # capa 1
    p.scatter(d["year"], d["lifeExp"], size=7,
             color=color, legend_label=país)    # capa 2

p.legend.location = "bottom_right"
p.legend.click_policy = "hide"  # clic → oculta la serie
```

Construir por capas

Una capa por glyph: p.line y p.scatter se superponen en el mismo lienzo

legend_label: crea la leyenda; misma etiqueta, mismo grupo

click_policy='hide': la leyenda aísla series

Paletas: bokeh.palettes (Category10, Viridis) evita elegir colores a mano

3. Bokeh: construir capa a capa

Selección vinculada (brushing) en Bokeh

```
# 01_04_02_bokeh_seleccion_vinculada.py
# 1. Una única fuente compartida para las dos vistas
source = ColumnDataSource(df)
herramientas =
    ("pan,wheel_zoom,box_select,lasso_select,reset,save")
propiedades = dict(
    source=source,
    color="color",
    selection_alpha=1.0,
    selection_line_color="black",
    nonselection_alpha=0.10,
)
# 2. Vista de PIB y esperanza de vida
p1 = figure(x_axis_type="log", tools=herramientas)
p1.scatter(x="gdpPercap", y="lifeExp", **propiedades)
# 3. Vista de población y esperanza de vida
p2 = figure(x_axis_type="log", y_range=p1.y_range,
            tools=herramientas)
p2.scatter(x="pop", y="lifeExp", **propiedades)
# El enlace funciona sin callback:
# ambas vistas comparten source.
diseno = row(p1, p2)
save(diseno)
```

Interacción coordinada

Fuente compartida: ambas vistas leen el mismo ColumnDataSource.

Brushing: lo seleccionado con caja o lazo se resalta en la otra.

nonselection_alpha: atenúa el resto para destacar la selección.

propiedades: mismo estilo visual en las dos vistas.

Rangos compartidos:

y_range=p1.y_range sincroniza zoom y pan.

3. Bokeh: construir capa a capa

Slider temporal con CustomJS en Bokeh

```
# 01_04_03_bokeh_slider_temporal.py
source_completo = ColumnDataSource(df[columnas])
source_visible = ColumnDataSource(df_inicial)

p = figure(x_axis_type="log",
           x_range=(100, 100_000),
           y_range=(25, 90))
p.scatter(x="gdpPercap", y="lifeExp",
          source=source_visible, size="tamano")

# Slider y callback ejecutado en navegador
slider = Slider(start=anios[0], end=anios[-1],
                value=anio_inicial, step=5, title="Año")
slider.js_on_change("value", CustomJS(
    args=dict(...),
    code="""
        // ...
        visible.data = salida;
        visible.change.emit();
    """,
))
diseno = column(slider, p)
```

Claves del slider

Dos fuentes: completo guarda el histórico; visible, lo dibujado

js_on_change + CustomJS: el filtrado se hace en JavaScript; el HTML funciona sin servidor

Escala estable: tamaño según el máximo histórico y ejes fijos

Ampliación, Bokeh Server: con bokeh serve los callbacks pueden escribirse en Python

3. Bokeh: construir capa a capa

Índice

1. Plotly Express: describir el gráfico
2. La figura Plotly por dentro
3. Bokeh: construir capa a capa
4. Elegir, practicar y seguir explorando

4. Elegir, practicar y seguir explorando

Plotly Express vs. Bokeh: criterios de selección

Cuadro comparativo

Rapidez de desarrollo: gana Plotly Express (menos líneas de código para resultados profesionales)

Personalización fina: gana Bokeh (más control sobre layout, leyendas y tooltips)

Integración web app: Plotly es nativo en Dash; Bokeh dispone de Bokeh Server

Mapas y 3D: Plotly integra soporte para mapas y gráficos 3D



Errores comunes (muro de los horrores)

Tooltips basura: nombres de columnas crudos como 'logged_gdp_pc' sin formato

Animaciones saltarinas: no fijar los rangos X/Y durante la reproducción temporal

Exceso de widgets: deslizadores innecesarios que confunden en lugar de guiar

Archivos HTML pesados: incluir datasets de millones de filas directamente en el cliente web



Explora ejemplos

Plotly

Open Source Python Graphing and Data Visualization Library

Plotly's Python graphing library makes interactive, publication-quality graphs. Examples of how to make line plots, scatter plots, area charts, bar charts, error bars, box plots, histograms, heatmaps, subplots, multiple-axes, polar charts, and bubble charts.

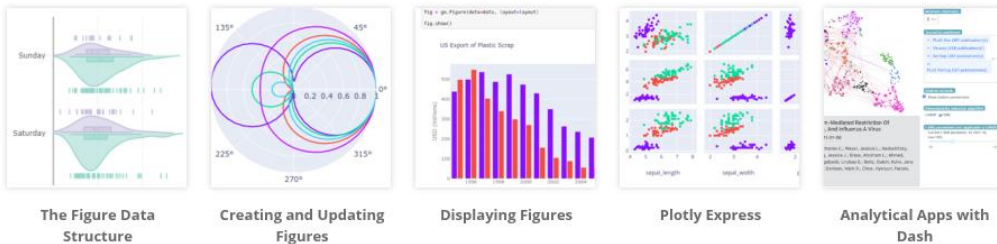
Plotly.py is free and open source, and you can [view the source](#), [report issues](#), or [contribute on GitHub](#).

Plotly Studio: Transform any dataset into an interactive data application in minutes with AI. [Try Plotly Studio now.](#)

Fundamentals

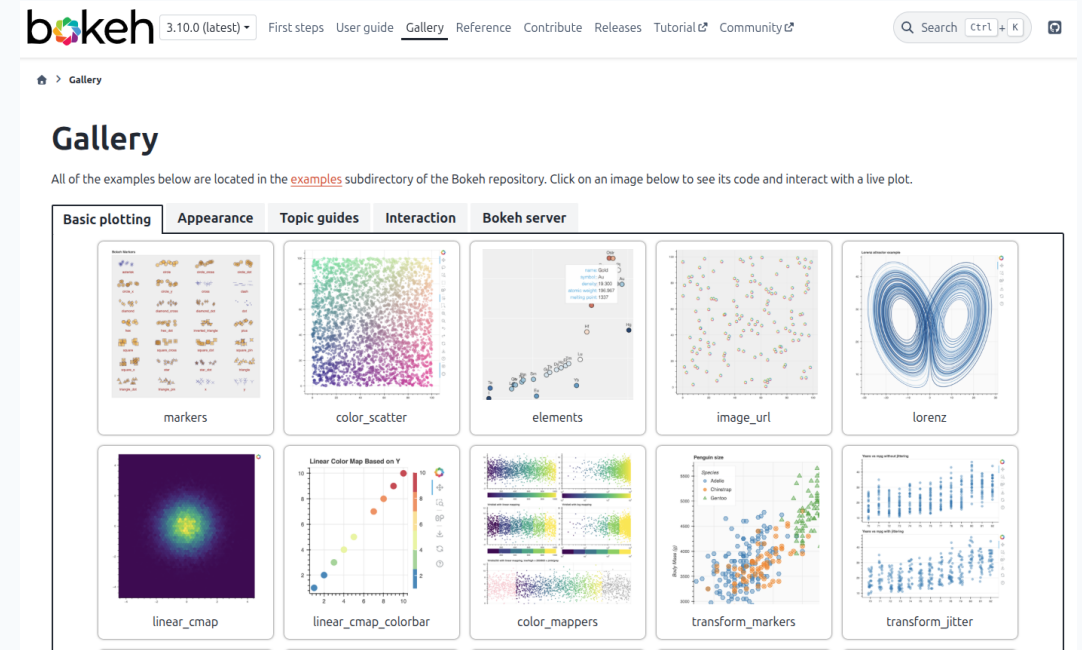
Explore step-by-step tutorials and code examples for figure structures, updating figures, displaying figures, and framework integrations to master our Python graphing library.

[More Fundamentals »](#)



<https://plotly.com/python/>

Bokeh



<https://docs.bokeh.org/en/latest/docs/gallery.html>

4. Elegir, practicar y seguir explorando

Tema 3

Plotly y Bokeh en la práctica

Jorge Dueñas Lerin

jorge.duenas.lerin@upm.es
